



DEEP LEARNING

AULA 1



Prof. Marco Antônio Simões Teixeira



CONVERSA INICIAL

Técnicas de *Deep Learning* estão sendo amplamente utilizadas e consideradas como o estado da arte em áreas de reconhecimento de padrões em imagens ou dados, como objetos em uma imagem e de padrões de comportamento para sistemas inteligentes e assistentes pessoais, entre diversas outras aplicações.

Nesta aula, estudaremos um pouco sobre a história da *Deep Learning*, buscando entender alguns de seus conceitos e tipos, além de praticar um pouco, utilizando a linguagem de programação Python e o *Framework MediaPipe*.

Ao final desta aula, esperamos atingir os seguintes objetivos, os quais serão avaliados ao longo dos estudos.

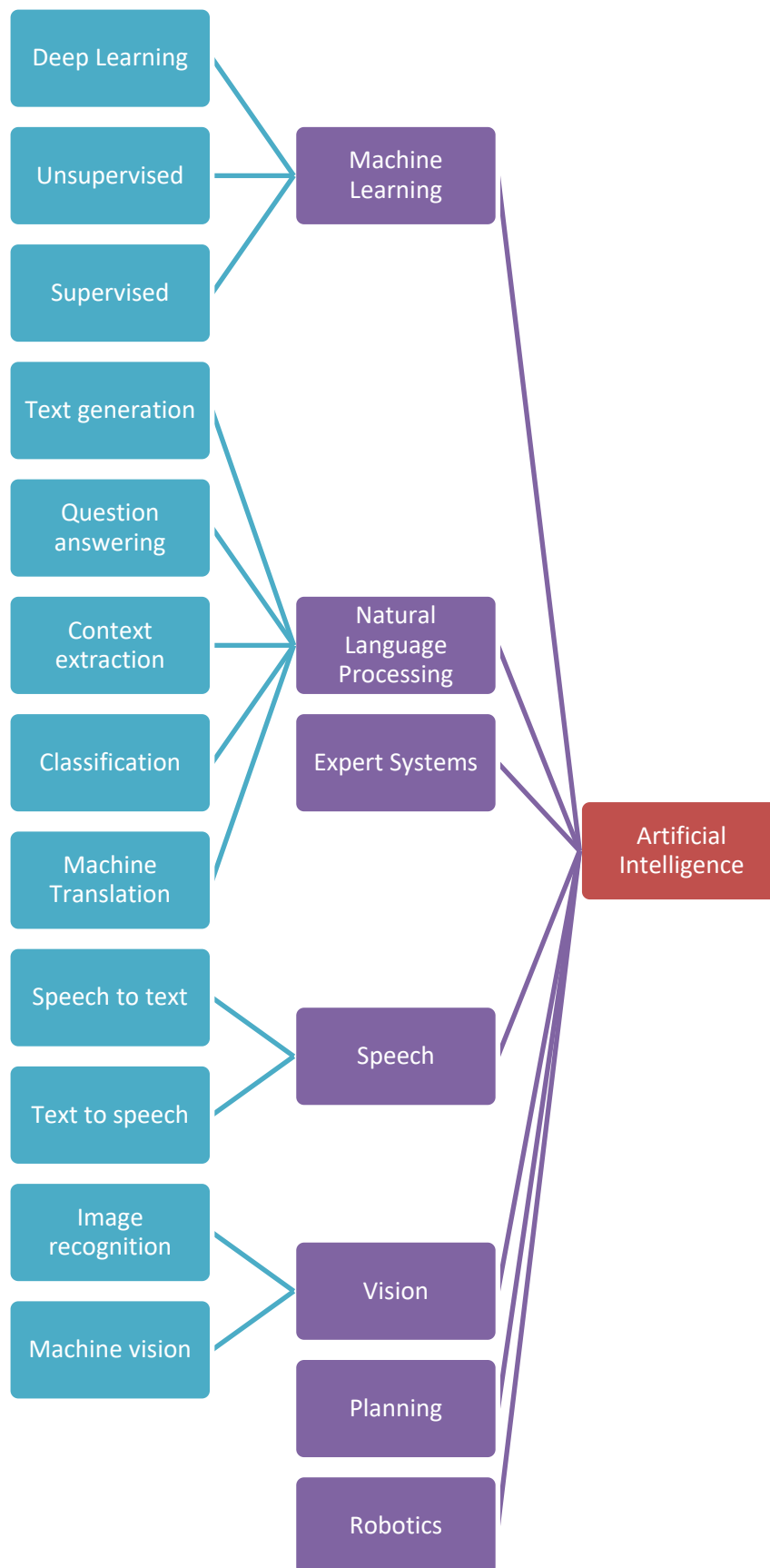
1. Conhecer um pouco da história da Inteligência Artificial, aprendizado de máquina e aprendizagem profunda.
2. Conhecer alguns conceitos de aprendizagem profunda, como suas estruturas, tipos e funcionalidades.
3. Praticar, executando uma ferramenta fácil e prática que nos forneça um resultado visual imediato.

TEMA 1 – UM POUCO SOBRE A HISTÓRIA

Primeiramente, é de fundamental importância que o conceito de *Deep Learning* seja compreendido, e que conheçamos sua história. A *Deep Learning*, ou “aprendizado profundo”, é uma subárea de *Machine Learning* (em português, “aprendizado de máquinas”), que, por sua vez, é uma subárea da inteligência Artificial, como apresentado na Figura 1.



Figura 1 – Inteligência Artificial e suas diferentes áreas



Fonte: Morisse, 2017.



A Inteligência Artificial (IA) pode ser vista como a grande área no que diz respeito a transferir para um equipamento a capacidade de resolver problemas tradicionalmente solucionados por pessoas. Segundo Sage (1990), a IA tem como objetivo a criação de algoritmos capazes de realizar tarefas cognitivas que hoje são mais bem executadas por humanos.

Dessa forma, a IA se torna generalista para todas as ferramentas e técnicas utilizadas para dar a um computador alguma capacidade cognitiva pelo uso de técnicas de aprendizado de máquina, ou por regras definidas, como o algoritmo de um jogo de xadrez.

Ainda de acordo com Sage (1990), um sistema de IA deve ser capaz de executar três tarefas principais: armazenar conhecimento, aplicar o conhecimento armazenado e adquirir novo conhecimento.

1.1 Um pouco da história da Inteligência Artificial

Conforme Coppin (2015), se levarmos em consideração o termo “inteligência” presente no termo Inteligência Artificial, sua origem é muito anterior aos computadores. Filósofos como Platão e Aristóteles criaram teorias e formularam questões com o objetivo de definir o que é inteligência. Tais teorias são utilizadas hoje de modo a tentar recriar os conceitos de inteligência existentes em equipamentos, como os computadores, por meio de algoritmos e códigos.

No entanto, é possível afirmar que, nos anos 1950, o termo começou a ser difundido como uma forma de criar um computador capaz de “pensar”. O primeiro artigo publicado nessa área foi “*Computing machinery and intelligence*”, de Alan Turing, em 1950.

Alan Turing participou da Segunda Guerra Mundial, quando trabalhou decifrando códigos alemães (recomendamos assistir ao filme *O jogo da imitação*). Ao final da guerra, ele decidiu dedicar-se à criação de computadores inteligentes, podendo, por isso, ser considerado um dos pioneiros na área da IA.

Turing chegou a desenvolver um teste para avaliar se o computador era, de fato, inteligente ou não. Chamado de **Teste de Turing**. O teste funciona da seguinte forma: uma pessoa irá interrogar um humano e um computador. O interrogador pode fazer perguntas aos interrogados, mas não pode interagir. Caso o interrogador fique em dúvida sobre quem é o humano, o equipamento terá sido aprovado no teste, sendo considerado inteligente.



Em virtude do teste, surgiram vários trabalhos com foco em simular uma conversa humana. À primeira vista, essa não é uma tarefa útil para um computador, mas trouxe avanços significativos para a computação, principalmente na área de processamento de linguagem natural, que é ainda estudada nos dias de hoje. Coppin (2015) informa que até o presente momento, nenhum computador foi capaz de passar pelo teste de Turing.

Nos anos 1960, a IA passou por uma reformulação. Os pesquisadores da época começaram a perceber que a tarefa de criar um equipamento inteligente não seria tão fácil como o esperado, e que seria necessário o uso de diferentes estratégias e técnicas para que fosse possível chegar a esse resultado.

Desse modo, surgiram algoritmos e heurísticas tentando imitar o comportamento do cérebro humano, como o **Analogia**, projetado por Thomas Evans, capaz de resolver problemas de analogia. Após a criação de técnicas para solucionar problemas de analogia, o otimismo com relação à IA voltou à tona, e novas técnicas e estudos foram desenvolvidos, dando origem a diversas áreas e subáreas dentro da IA, como aprendizado de máquina, sistemas multiagentes, vida artificial, visão por computador, planejamento, jogos, entre outras.

TEMA 2 – REDES NEURAIS ARTIFICIAIS E A DEEP LEARNING

Estudaremos agora sobre o surgimento do *Deep Learning*. Para isso, é preciso conhecer a história das Redes Neurais Artificiais. Nesta aula, estudaremos sua história para, posteriormente, tratarmos da visão técnica e assim entendermos seu conceito e funcionamento.

2.1 Redes Neurais Artificiais

A criação do conceito de Redes Neurais teve início com o trabalho do psiquiatra e neuroanatomista Warren McCulloch com o prodígio matemático Walter Pitts. Em 1943, eles publicaram o trabalho inaugural do estudo das Redes Neurais modernas, em uma conferência sobre modelagem neural na *University of Chicago*.

Na época, o objetivo não era criar modelos computacionais, mas modelos matemáticos que representassem o sistema nervoso humano – dado que os trabalhos de McCulloch e Pitts foram desenvolvidos antes mesmo dos trabalhos



de Alan Turing. Este artigo de 1943 foi popular na época e continua sendo ainda hoje.

Saiba mais

Se você tiver interesse em ler o famoso artigo “*A logical calculus of the ideas immanent in nervous activity*”, de McCulloch e Pitts, acesse o link a seguir.

Disponível em <<https://link.springer.com/article/10.1007/BF02478259>>. Acesso em: 10 fev. 2022.

A partir do trabalho de McCulloch e Pitts, sugeriram muitos outros, criando variantes e novos modelos de neurônios e técnicas de aprendizado. Por exemplo, em 1986, von Neymann usou chaves de atraso derivadas do neurônio de McCulloch e Pitts (Spray; Burks, 1986).

Em 1948, o livro *Cybernetics*, de Wiener, trouxe alguns conceitos de controle e processamento de sinais, entre outras aplicações, utilizando as Redes Neurais. Em sua segunda edição (1961), foram adicionados conceitos de aprendizagem e auto-organização, preparando terreno para a criação de modelos artificiais de Redes Neurais. Entretanto, somente depois de mais de 30 anos, Hopfield desenvolveu a ligação entre mecânica estatística e os sistemas de aprendizagem.

Em 1949, Hebb publicou o livro *Organization of Behavior*, no qual foi apresentada, pela primeira vez, uma formulação para a aprendizagem fisiológica para a modificação sináptica. Depois do livro de Hebb, começaram a surgir modelos computacionais tentando criar sistemas adaptativos e de aprendizagem.

É possível dizer que o trabalho de Holland, Haibt e Duda, publicado em 1956, deu início à conversão dos modelos neurais em simulação computacional, quando percebeu-se que algumas alterações necessitavam ser feitas, como a criação de inibidores. Com base nesse trabalho e com o avanço da computação, mais e mais modelos computacionais surgiram, como o de Uttley (1956), que comprovou que é possível modificar as sinapses para aprender a classificar conjuntos binários.

Depois desses trabalhos, o tema evoluiu. Podemos avançar alguns anos e nos concentrarmos nos trabalhos relevantes, como o de Rosenblatt (1958), que, 15 anos após o artigo de McCulloch e Pitts, propôs o teorema da



Convergência do Perceptron, que é utilizado até hoje para o treinamento de Redes Neurais Artificiais.

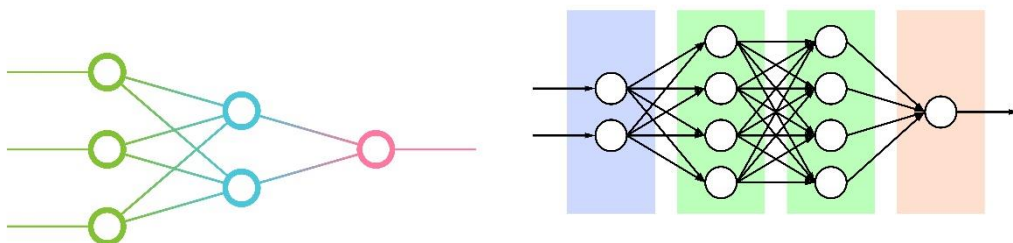
No entanto, em 1969, foi publicado o livro de Minsky e Papert, que demonstra, de forma matemática, as limitações nos Perceptrons de camada única, mas também sugere que não existem razões para crer que os Perceptrons de múltiplas camadas possam superá-los. Devemos aqui fazer uma ressalva: o termo *Deep Learning* vem do conceito de redes profundas; dessa maneira, as redes Perceptrons de múltiplas camadas podem ser consideradas o início da *Deep Learning*. Até este momento da história, o conceito é somente teórico.

Após o livro de Minsky e Papert, além das limitações computacionais da época, o estudo em modelos neurais computacionais desacelerou, com poucos pesquisadores dedicando-se ao tema. Somente nos anos 1980, mais de 10 anos após a publicação do livro, surgiram avanços significativos no campo.

Em 1982, Hopfield transformou a área das Redes Neurais, utilizando a ideia de função de energia para entender e modelar os sistemas computacionais que executam redes recorrentes sinápticas. Esse fato fez com que físicos teóricos e práticos de centros de pesquisa de todo o mundo se interessassem novamente pelo assunto, o que ainda persiste.

A Figura 2 traz exemplos, respectivamente, de Redes Neurais Artificiais de uma camada, e Redes Neurais Artificiais de múltiplas camadas, representando graficamente o que podemos considerar a *Deep Learning*.

Figura 2 – À esquerda, Redes Neurais Artificiais de camada única; à direita, Redes Neurais Artificiais de múltiplas camadas



Créditos: all_is_magic/Shutterstock.

TEMA 3 – UM POUCO SOBRE OS CONCEITOS DE *DEEP LEARNING*

Agora que conhecemos um pouco sobre a história da IA e como ela deu origem à *Deep Learning*, analisaremos, de forma simplificada, o que seria a *Deep*



Learning, seus conceitos e aplicações. Primeiro, conheceremos a diferença entre os conceitos de *Deep Learning* e *Machine Learning*.

Machine Learning, ou “aprendizado de máquina”, é um termo utilizado para estratégias que inferem conhecimento em algoritmos computacionais, como Redes Neurais Artificiais e a *Deep Learning*. No entanto, por se tratar de um termo que engloba várias técnicas, ele é mais genérico e permite a utilização de estratégias diferentes para chegar ao mesmo fim.

Na Figura 3, os dois algoritmos têm o mesmo objetivo: identificar um carro. Se a identificação for desenvolvida com técnicas tradicionais de *Machine Learning*, a imagem passará por um *pipeline* com vários algoritmos de processamento de imagem tradicionais, como filtros de detecção de contornos, redução de ruídos, entre outros, e, no final, este seria aplicado a um classificador (normalmente uma Rede Neural Artificial totalmente conectada) para tentar abstrair as informações de que, naquela imagem há um carro, com base nos dados já processados pelo *pipeline*.

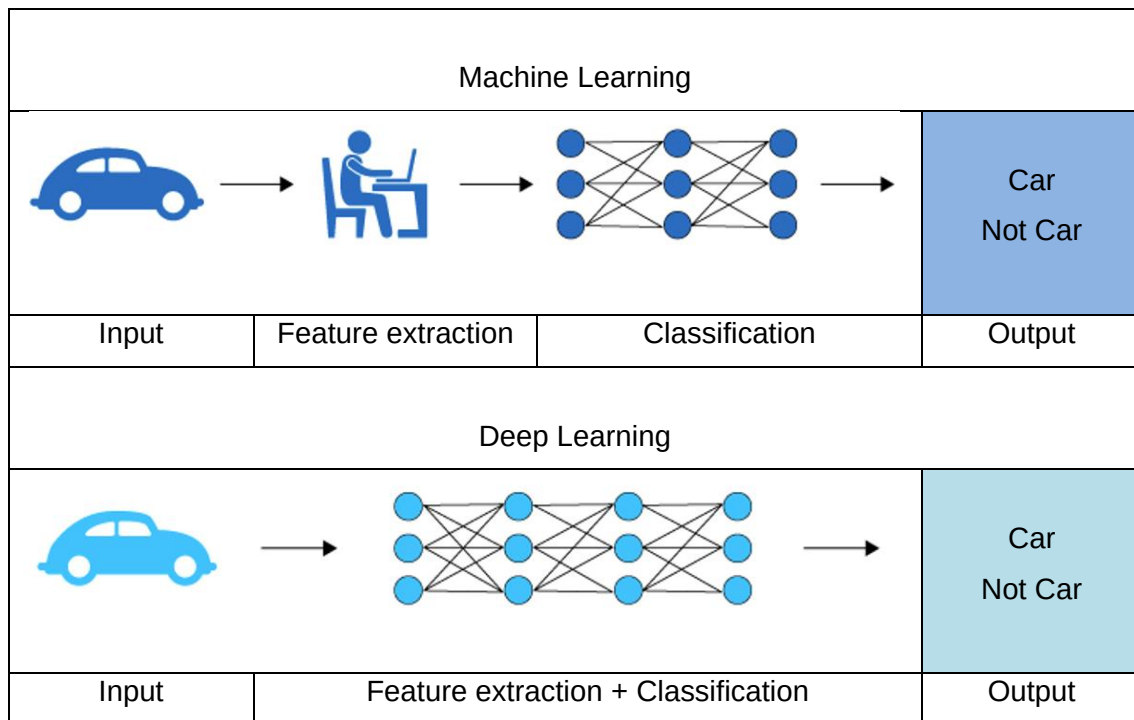
Já na *Deep Learning*, o objetivo é treinar não somente o classificador final, mas todo o *pipeline* que normalmente é desenvolvido por algoritmos específicos. Em resumo, você treina também um extrator de características, que será adicionado a um classificador.

A vantagem de treinar todas as etapas do reconhecimento são inúmeras, como o tempo de execução. Utilizar uma *Deep Learning*, em muitos casos, é mais leve do que utilizar técnicas de *pipeline* associadas a um classificador, e os avanços nessa área não param.

Cada dia há soluções mais leves e robustas, que podem ser executadas em *smartphones* e computadores domésticos com pouco poder de processamento. Outra vantagem é a taxa de acerto: utilizando uma rede treinada do início ao fim em conjunto, os resultados são superiores aos obtidos com técnicas tradicionais, com o uso de *pipeline* e classificadores simples.



Figura 3 – Exemplo simples de diferença entre *Deep Learning* e *Machine Learning*



Fonte: Lawtomated, 2019.

TEMA 4 – EXEMPLOS DE ESTRUTURAS DE *DEEP LEARNING*

Vamos agora estudar um pouco mais sobre *Deep Learning*. Dentro da *Deep Learning*, há uma variedade de técnicas e formatos de redes existentes. Existem, por exemplo, as redes neurais tradicionais com múltiplas camadas, comumente conhecidas como *Multi-layer Perceptron* (MLP); as *Convolutional Neural Networks* (CNN), ou, em português, “Redes Neurais Convolucionais”.

Há ainda a *Recurrent Neural Network* (RNN) e técnicas como a *Modular Neural Network*, cujo objetivo é unir diferentes tipos de redes como se fossem blocos para uma solução final. Aqui, vamos focar em três tipos de redes: as MLP, CNN e RNN; abordaremos também técnicas de transferência do conhecimento. Vamos estudar um pouco mais sobre estas redes.

4.1 *Convolutional neural networks* (CNN)

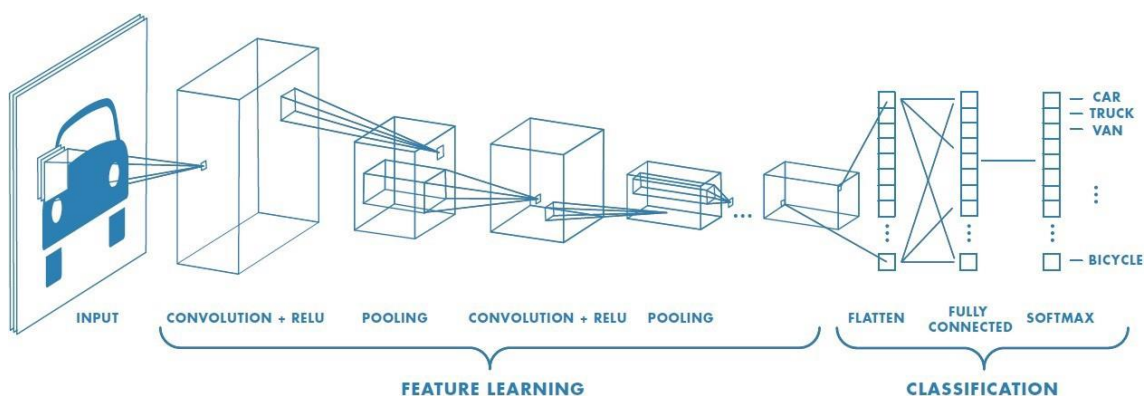
Uma das técnicas de *Deep Learning* mais utilizadas consiste em treinar tanto o extrator de características quanto o classificador na ponta da rede. A



Figura 4 traz um exemplo da arquitetura de uma CNN. Observe que a imagem é adicionada à rede, e então passa por várias camadas.

Cada camada possui uma função diferente; algumas, têm a função de reduzir a imagem; outras, de aumentar; algumas aplicam máscaras e filtros para tentar extrair características não perceptíveis ao olho humano. No final, essas características são adicionadas a um classificador para que se possa dizer que, com base naquelas informações retiradas da imagem, há ou não uma pessoa, por exemplo.

Figura 4 – Exemplo de uma arquitetura de *Deep Learning*



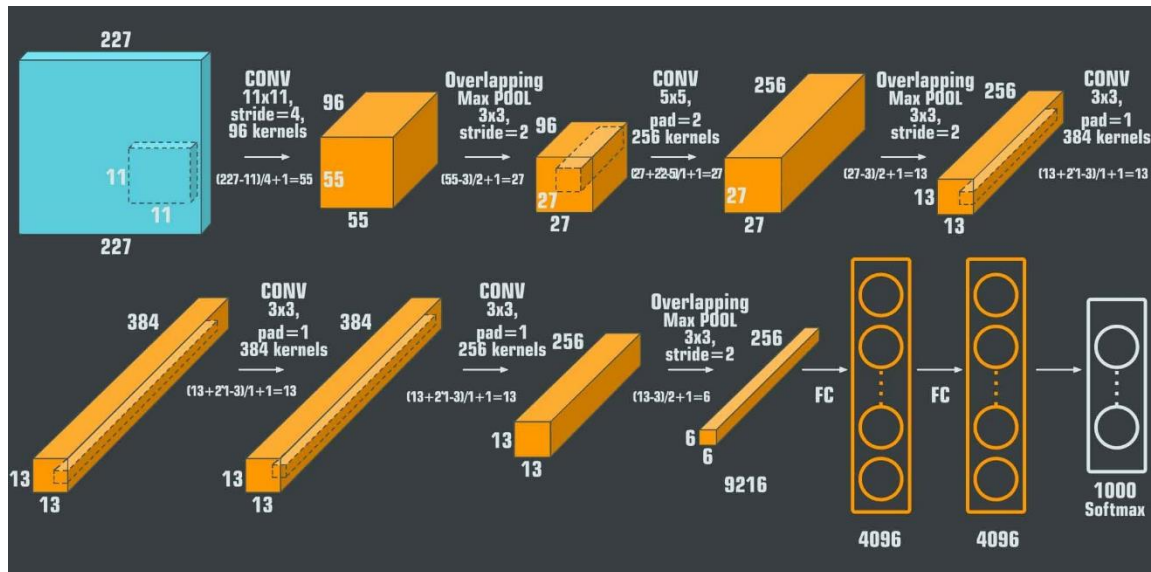
Fonte: Run AI, 2021.

Há diferentes modelos de CNN, alguns famosos, como o caso da AlexNet (Figura 5). Note que a rede é formada por diferentes camadas, como camadas convolucionais, *Max Pooling*, entre outras. Ao final, a imagem passa por uma rede totalmente conectada para realizar a classificação.

Além da AlexNet, existem diversos outros modelos consagrados pela literatura. Cada um deles visa resolver um tipo de problema diferente e, normalmente, é associado a um artigo acadêmico que o descreve. Com relação às camadas citadas no interior da rede, iremos estudá-las futuramente na ocasião sobre redes CNN. No entanto, deixamos aqui a sugestão para você realizar uma pesquisa rápida sobre as redes CNN conhecidas e suas aplicações.



Figura 5 – Arquitetura de uma das redes CNN mais conhecidas, a AlexNet



Crédito: Shadedesign/Shutterstock.

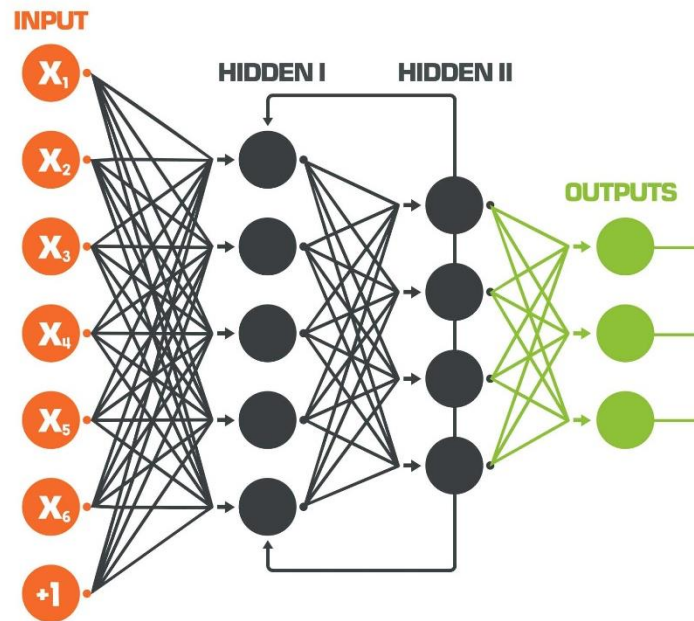
Exemplos de usos de CNN podem envolver diferentes áreas do conhecimento e aplicações. A CNN é comumente utilizada para, por exemplo, classificar fotos de forma inteligente. Imagine que você possua um álbum de fotos, todas fora de ordem. Uma rede CNN pode ser utilizada para separar as fotos em álbuns, como “fotos de pessoas”, “fotos de paisagens”, “fotos de animais” etc.

A CNN também pode ser utilizada para tarefas de engenharia, como a automação industrial. É possível criar uma rede que identifique pessoas em locais proibidos e faça com que os equipamentos autônomos parem de operar, evitando que aquela pessoa seja ferida. São muitas as aplicações para as redes CNN.

4.2 Recurrent Neural Network (RNN)

A RNN utiliza algumas saídas de camadas intermediárias como entrada, criando uma espécie de realimentação. Essa técnica apresenta algumas vantagens, como conseguir abstrair padrões no tempo, e não somente em dados estáticos. Isso ocorre porque a realimentação pode ser feita com o atraso de tempo em relação à entrada.

Figura 6 – Exemplo simplificado da representação de uma *Recurrent Neural Network* (RNN)



Crédito: Shadedesign/Shutterstock.

TEMA 5 – EXEMPLOS E USOS DE FERRAMENTAS DE *DEEP LEARNING*

O objetivo desta aula é apresentar o histórico e os conceitos de *Deep Learning* de forma geral, sem entrarmos no mérito de como criar e treinar uma rede “do zero”. Conhecemos um pouco sobre sua história e também as técnicas de *Deep Learning* existentes. Estudaremos, posteriormente, essas técnicas mais aprofundadamente.

Nesse momento, para conhecermos mais sobre o potencial da ferramenta, executaremos alguns exemplos de redes já treinadas, utilizando o Python.

Uma das bibliotecas que está se destacando nos últimos tempos é a *MediaPipe*. É uma biblioteca desenvolvida pelo Google, e é utilizada nas ferramentas da empresa, como o Google Meet, usada para identificar o corpo de uma pessoa e modificar o fundo. Essa biblioteca está disponível para diversas plataformas e linguagens, podendo ser utilizada tanto em dispositivos móveis com o Android, quanto em códigos em Python (que é o nosso caso).

O *MediaPipe* pode ser considerado um *framework* que concentra diferentes ferramentas em um único lugar. Por exemplo, com o *MediaPipe* é possível reconhecer objetos, identificar contornos de pessoas, identificar pontos no rosto, entre outras atividades, utilizando a mesma biblioteca.

Entretanto, cada ação se refere a uma estratégia diferente. A maioria delas são de redes CNN, mas que não se limitam ao uso de *Deep Learning*.



Algumas de suas soluções utilizam técnicas tradicionais de processamento de imagem. Desse modo, cada aplicação do *MediaPipe* usa técnicas diferentes, e essa ação é transparente ao usuário, que instala e utiliza uma única biblioteca.

5.1 Instalando o *MediaPipe*

A vantagem do *MediaPipe* é sua facilidade de uso e instalação. Ele pode ser instalado pelo pip, que é um sistema de gerenciamento de pacotes, que contempla diversas bibliotecas do Python, facilitando o trabalho com diferentes plataformas. Por exemplo: o mesmo comando que é utilizado para instalar a biblioteca em um computador com Windows, é utilizado para um computador com Linux.

Configurações do sistema utilizado nos testes realizados a seguir:

- Sistema operacional Ubuntu 20.04;
- Python 3.8.10; e
- Pip 20.0.2.

Não abordaremos a instalação do pip, visto que ela pode ser diferente para cada sistema operacional. Como referência, os passos de instalação para cada sistema operacional podem ser encontrados na página oficial do Pip¹.

Considerando que o pip esteja corretamente instalado, basta abrir o terminal para instalar o *MediaPipe*, indiferentemente do sistema operacional utilizado e digitar:

```
'''  
$ pip install MediaPipe  
'''
```

O pip gerenciará e instalará todos os pacotes necessários para a utilização correta do *MediaPipe*. Feito isto, a biblioteca já está instalada em sua máquina e pronto para uso.

5.2 *MediaPipe Face Detection*

Como mencionado anteriormente, o *MediaPipe* é composto de várias técnicas diferentes. O *MediaPipe Face Detection* é capaz de identificar o rosto

¹ Disponível em: <<https://pip.pypa.io/en/stable/installation/>>. Acesso em: 10 fev. 2022.



de uma pessoa. Essa ação é útil para diferentes aplicações, como identificar o rosto, para então identificar se a pessoa está, ou não, utilizando máscara.

Esse programa utiliza a rede BlazeFace, que é introduzida no artigo “*BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs*”²..

Vamos à prática?

Após a instalação do *MediaPipe*, é possível desenvolver o código que irá executar o *MediaPipe Face Detection*.

''' Início código

```
1. import cv2
2. import MediaPipe as mp
3. mp_face_detection = mp.solutions.face_detection
4. mp_drawing = mp.solutions.drawing_utils
5.
6.
7. # For webcam input:
8. cap = cv2.VideoCapture(0)
9. with mp_face_detection.FaceDetection(
10.     model_selection=0, min_detection_confidence=0.5) as face_detection:
11.     while cap.isOpened():
12.         success, image = cap.read()
13.         if not success:
14.             print("Ignorando frames em branco.")
15.             continue
16.
17.         #Conversão da imagem no padrão openCV RGB
18.         image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
19.         #Aplicando a imagem a rede
20.         results = face_detection.process(image)
21.
22.         #Desenhando os rostos detectados na imagem
23.         image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
24.         if results.detections:
25.             for detection in results.detections:
26.                 mp_drawing.draw_detection(image, detection)
27.
28.         # Apresentando o resultado
29.         cv2.imshow('MediaPipe Face Detection', cv2.flip(image, 1))
30.         if cv2.waitKey(5) & 0xFF == 27:
31.             break
```

² Sugerimos a leitura do artigo a seguir ou, pelo menos, a visualização do modelo da rede, presente na Figura 1 do artigo. Disponível em: <<https://arxiv.org/abs/1907.05047>>. Acesso em: 10 fev. 2022.



```
32. cap.release()
'''Fim código
```

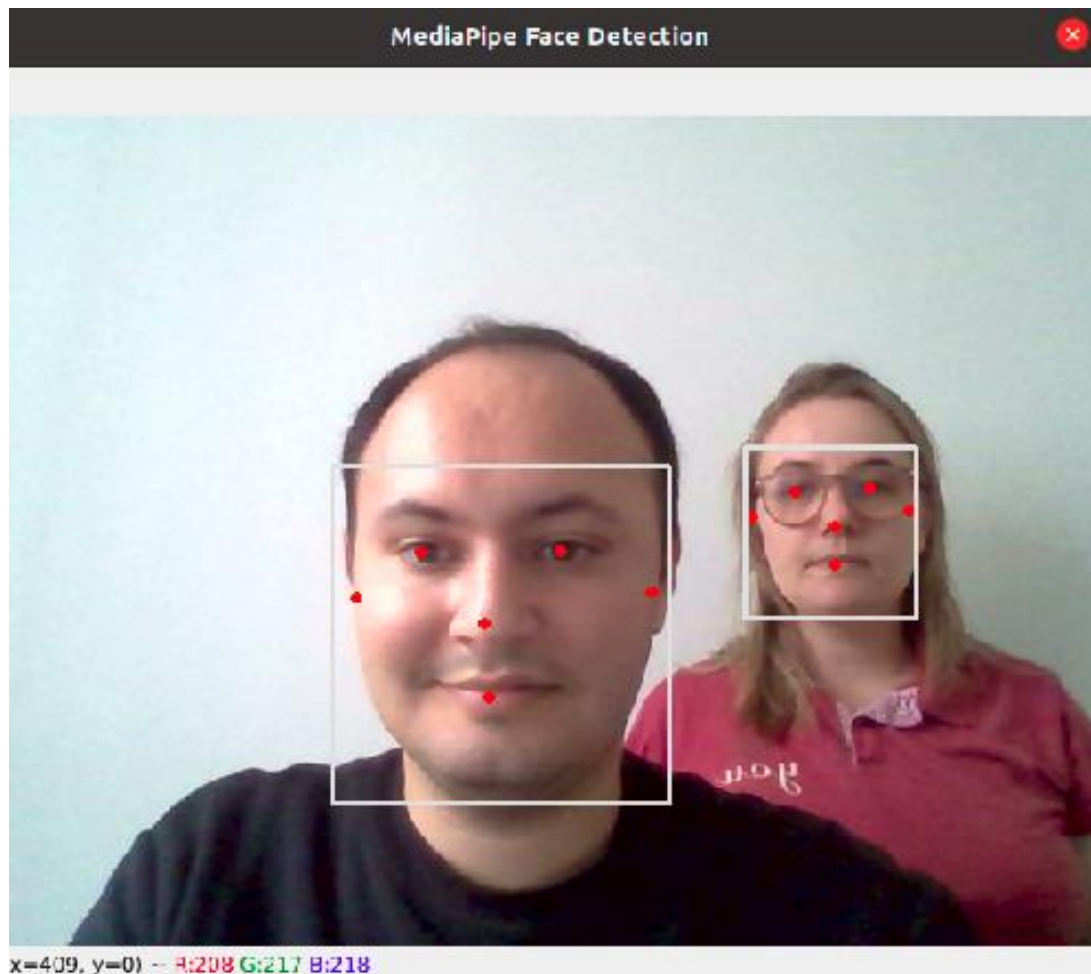
Vamos à análise do código. Das linhas 1 a 4, estão sendo realizadas as importações das bibliotecas necessárias para a execução do *MediaPipe Face Detection*. A linha 8 inicia a captura de imagem da *webcam*. A linha 9 dá início ao *loop* de execução do código. A linha 20 é a responsável por aplicar a imagem ao *MediaPipe*. Veja que, em uma única linha, a imagem é adicionada ao *framework*, e tem como retorno os dados de rostos detectados.

Nesse momento, os resultados são apenas números contendo a posição dos rostos; já nas linhas 23 a 26, é aplicado o resultado à imagem original e, pôr fim, a imagem é apresentada.

Para executar o código, salve-o em um arquivo com a extensão “.py”, como “exemplo1.py”. Para executar o código, basta navegar pelo terminal até onde ele está salvo e, então, executar o comando que será apresentado a seguir. O resultado pode ser visto pela Figura 7, em que os rostos de duas pessoas são identificados. Vale ressaltar que o *MediaPipe* utiliza o CPU por padrão, podendo ser optado pela GPU (ganho de performance).

```
'''
$ python3 exemplo1.py
'''
```


Figura 7 – *MediaPipe Face Detection* em funcionamento

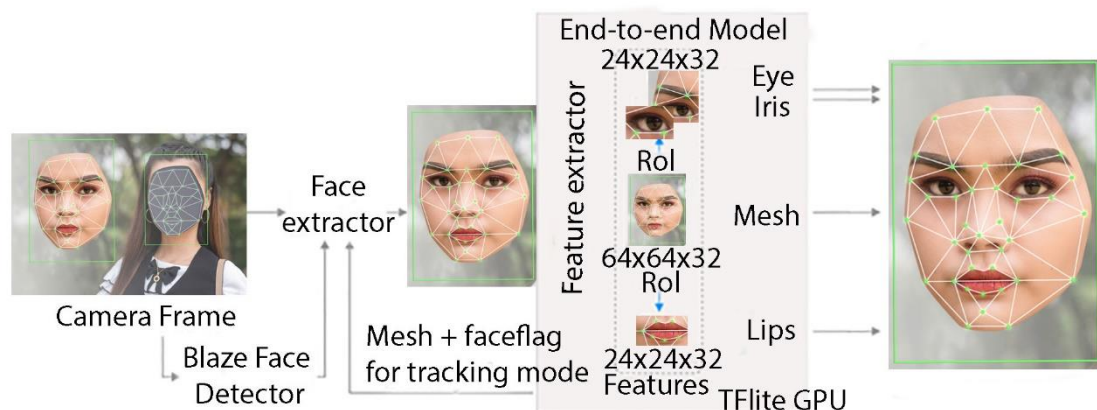


Crédito: Marco Antônio Simões Teixeira.

5.3 *MediaPipe Face Mesh*

Como mencionado, o *MediaPipe* utiliza outras técnicas além de *Deep Learning*. Um exemplo é o *MediaPipe Face Mesh*. Esta abordagem utiliza o mesmo algoritmo para detecção de rosto (*BlazeFace*) e então executa técnicas de pós-processamento para identificar mais objetos no rosto, criando uma máscara. A abordagem adotada pelo *MediaPipe Face Mesh* é apresentada no artigo “*Attention Mesh: High-fidelity Face Mesh Prediction in Real-time*”, e uma visão geral de seu funcionamento é apresentada na Figura 8.

Figura 8 – Arquitetura da *MediaPipe Face Mesh*



Fonte: Grishchenko, Ivan, et al. 2020.

Observe que o *MediaPipe Face Mesh* utiliza a saída do *BlazeFace* (*MediaPipe Face Detection*) e, então, adiciona a imagem identificada em um extrator de características, para então criar a máscara, realizando uma combinação de técnicas para alcançar o objetivo desejado. Esse tipo de abordagem é normal na área e, por isso, é interessante conhecer o maior número possível de ferramentas, para que, com elas, seja possível resolver o seu problema específico.

Vamos ao código: de modo geral, ele não difere muito do que vimos anteriormente, uma vez que o *MediaPipe* tende a utilizar um padrão em suas aplicações. Podemos notar como diferença importante as linhas 27 a 52. Como o *MediaPipe Face Mesh* retorna uma quantidade muito grande de pontos, é chamada uma ferramenta específica do *MediaPipe* para o desenho dos pontos na imagem: o “draw_landmarks”, pelo qual é passada a imagem original e os pontos; então, a função realiza o desenho dos pontos na imagem.

''' Código Python exemplo2.py

```
1. import cv2
2. import MediaPipe as mp
3. mp_drawing = mp.solutions.drawing_utils
4. mp_drawing_styles = mp.solutions.drawing_styles
5. mp_face_mesh = mp.solutions.face_mesh
6.
7.
8. # For webcam input:
9. drawing_spec = mp_drawing.DrawingSpec(thickness=1, circle_radius=1)
10. cap = cv2.VideoCapture(0)
11. with mp_face_mesh.FaceMesh(
```



```
12. max_num_faces=1,
13. refine_landmarks=True,
14. min_detection_confidence=0.5,
15. min_tracking_confidence=0.5) as face_mesh:
16. while cap.isOpened():
17.     success, image = cap.read()
18.     if not success:
19.         print("Ignorando frames em branco.")
20.         continue
21.
22.     #Conversão da imagem no padrão openCV RGB
23.     image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
24.     #Aplicando a imagem a rede
25.     results = face_mesh.process(image)
26.
27.     #Desenhando os rostos detectados na imagem
28.     image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
29.     if results.multi_face_landmarks:
30.         for face_landmarks in results.multi_face_landmarks:
31.             mp_drawing.draw_landmarks(
32.                 image=image,
33.                 landmark_list=face_landmarks,
34.                 connections=mp_face_mesh.FACEMESH_TESSELATION,
35.                 landmark_drawing_spec=None,
36.                 connection_drawing_spec=mp_drawing_styles
37.                 .get_default_face_mesh_tesselation_style())
38.             mp_drawing.draw_landmarks(
39.                 image=image,
40.                 landmark_list=face_landmarks,
41.                 connections=mp_face_mesh.FACEMESH_CONTOURS,
42.                 landmark_drawing_spec=None,
43.                 connection_drawing_spec=mp_drawing_styles
44.                 .get_default_face_mesh_contours_style())
45.             mp_drawing.draw_landmarks(
46.                 image=image,
47.                 landmark_list=face_landmarks,
48.                 connections=mp_face_mesh.FACEMESH_IRISES,
49.                 landmark_drawing_spec=None,
50.                 connection_drawing_spec=mp_drawing_styles
51.                 .get_default_face_mesh_iris_connections_style())
52.
53.     # Apresentando o resultado
54.     cv2.imshow('MediaPipe Face Mesh', cv2.flip(image, 1))
55.     if cv2.waitKey(5) & 0xFF == 27:
```

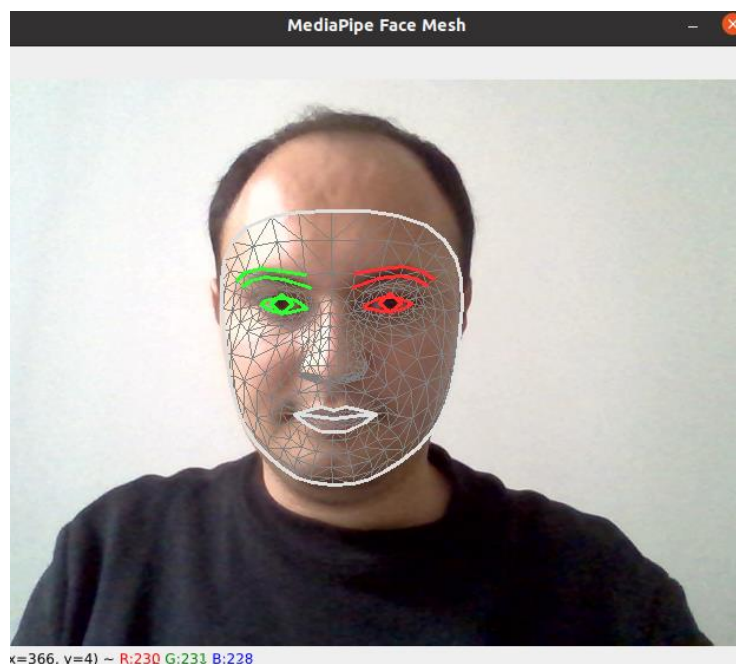
```
56.     break
57. cap.release()
```

```
""" Fim código
```

Para executarmos o código, como aconteceu no exemplo anterior, vamos salvá-lo em um arquivo “.py” e então executá-lo. Vamos utilizar o nome de “exemplo2.py”. Para executá-lo, basta digitar o código a seguir. Um exemplo do resultado pode ser visto na Figura 9.

```
'''
python3 exemplo2.py
'''
```

Figura 9 – Exemplo de execução do *MediaPipe Face Mesh*



Crédito: Marco Antônio Simões Teixeira.

5.3 *MediaPipe Pose*

Seguindo o que foi apresentado nas seções anteriores, o *MediaPipe Pose* é uma solução capaz de identificar o corpo de uma pessoa. Para isso, ele utiliza o “BlazePose”. Sobre este assunto, consulte o artigo “*BlazePose: On-device Real-time Body Pose tracking*”. Vamos ao código.



''' Início código

```
1. import cv2
2. import MediaPipe as mp
3. mp_drawing = mp.solutions.drawing_utils
4. mp_drawing_styles = mp.solutions.drawing_styles
5. mp_pose = mp.solutions.pose
6.
7.
8. cap = cv2.VideoCapture(0)
9. with mp_pose.Pose(
10.     min_detection_confidence=0.5,
11.     min_tracking_confidence=0.5) as pose:
12.     while cap.isOpened():
13.         success, image = cap.read()
14.         if not success:
15.             print("Ignorando frames em branco.")
16.             continue
17.
18.         #Conversão da imagem no padrão openCV RGB
19.         image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
20.         #Aplicando a imagem a rede
21.         results = pose.process(image)
22.
23.         #Desenhando os rostos detectados na imagem
24.         image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
25.         mp_drawing.draw_landmarks(
26.             image,
27.             results.pose_landmarks,
28.             mp_pose.POSE_CONNECTIONS,
29.             landmark_drawing_spec=mp_drawing_styles.get_default_pose_landmarks_style())
30.
31.         # Apresentando o resultado
32.         cv2.imshow('MediaPipe Pose', cv2.flip(image, 1))
33.         if cv2.waitKey(5) & 0xFF == 27:
34.             break
35. cap.release()
```

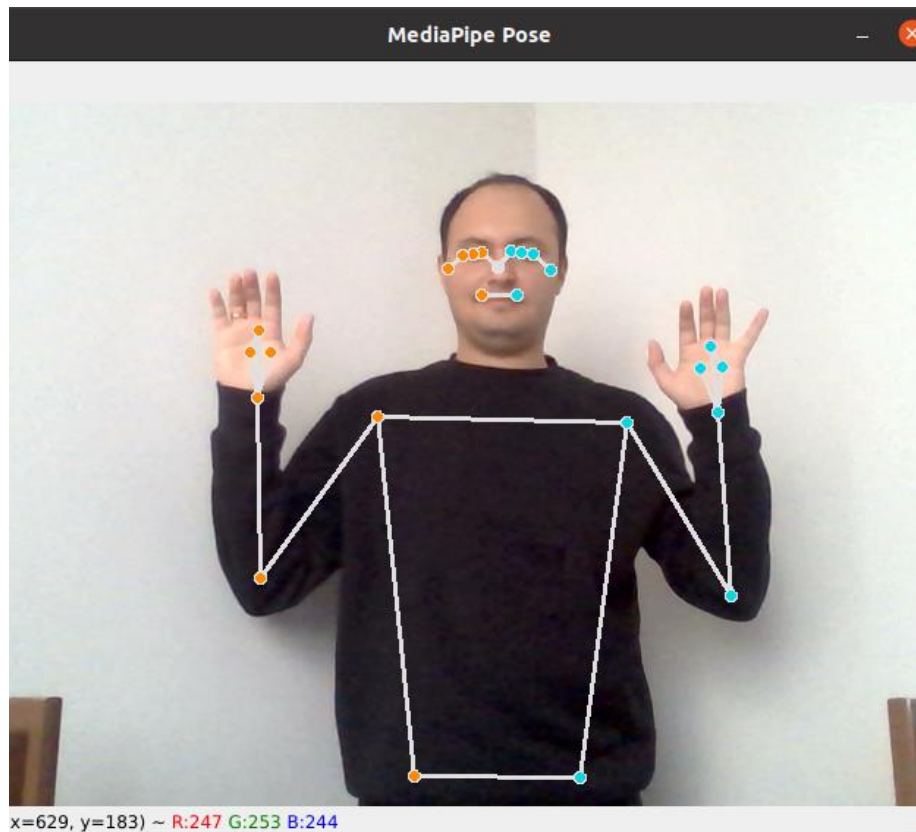
''' Fim código

Note que a estrutura é a mesma dos exemplos anteriores: primeiro, é coletada a imagem; então, aplicada a biblioteca e, em seguida, o resultado é desenhado na imagem original e apresentada ao usuário. Vamos salvar esse código como “exemplo3.py”. Para executá-lo, digite o código a seguir, e o resultado poderá ser observado na Figura 10.



```
'''  
python3 exemplo3.py  
'''
```

Figura 10 – Imagem aplicada ao *MediaPipe Pose*



Crédito: Marco Antônio Simões Teixeira.

Além dos exemplos apresentados até agora, o *MediaPipe* possui diversas outras ferramentas que podem ser acessados diretamente do *site* do *MediaPipe*. Aconselhamos a visitar o *site* e tentar executar os exemplos presentes, pois acreditamos que agora você é capaz de realizar essa tarefa.

FINALIZANDO

Nesta aula, demos início ao tema deste curso, estudando um pouco sobre a história da Inteligência Artificial e sobre os conceitos de aprendizagem profunda, como a sua diferença em relação às Redes Neurais Artificiais tradicionais de uma única camada, em que os dados são pré-processados antes da classificação.

Vimos também um exemplo de *framework* em Python, o qual utiliza redes já treinadas para realizar o processamento de imagens e identificar partes do



corpo. Futuramente, estudaremos de maneira mais técnica como as técnicas de *Deep Learning* funcionam, além de criarmos redes do zero e as treinarmos utilizando imagens disponíveis em banco de dados na internet.



REFERÊNCIAS

- A.I. TECHNICAL: Machine vs Deep Learning. **Lawtomed**, 28 abr. 2019. Disponível em: <<https://lawtomed.com/a-i-technical-machine-vs-deep-learning/>>. Acesso em: 9 fev. 2022.
- ASPRAY, W.; BURKS, A. (Ed.). **Papers of John von Neumann on computing and computer theory**. Cambridge: MIT Press, 1986.
- BAZAREVSKY, V. et al. **Blazeface**: Sub-Millisecond Neural Face Detection On Mobile GPUS. **ArXiv** abs/1907.05047, 2019).
- COPPIN, B. **Inteligência artificial**. Barueri: Grupo Gen-LTC, 2015.
- DEEP Convolutional Neural Networks: A Guide. **Run AI**, 2021. Disponível em: <<https://www.run.ai/guides/deep-learning-for-computer-vision/deep-convolutional-neural-networks>>. Acesso em: 9 fev. 2022.
- GRISHCHENKO, I. et al. **Attention Mesh**: High-fidelity Face Mesh Prediction in Real-time. 2020.
- HOPFIELD, J. J. Neural networks and physical systems with emergent collective computational abilities. **Proceedings of the national academy of sciences**, v. 79, n. 8, p. 2554-2558, 1982.
- KETKAR, N. **Deep learning with python, a hands-on introduction, a press edition**. 2017.
- MINSKY, M.; PAPERT, S. **Perceptrons**: An introduction to computational geometry. Cambridge: HIT, 1969.
- MORISSE, T. AI for Dummies. **Faber Novel**, Feb 23, 2017. Disponível em: <<https://www.fabernovel.com/en/article/tech-en/ai-for-dummies>>. Acesso em: 8 fev. 2022.
- ROCHESTER, N. et al. Tests on a cell assembly theory of the action of the brain, using a large digital computer. **IRE Transactions on information Theory**, 2.3 (1956), 80-93.
- ROSENBLATT, F. The perceptron: a probabilistic model for information storage and organization in the brain. **Psychological review**, 65.6, (1958), 386.
- SAGE, A. P. (Ed.). **Concise Encyclopedia of Information Processing in Systems & [and] Organizations**. New York: Pergamon Press, 1990.



TURING, A. M. Computing machinery and intelligence. In: PARSING the turing test. Dordrecht: Springer, 2009. p. 23-65.